

Table of Content

Introduction	2
Project Objective	2
Background	2
Motor types.....	2
Brushed motor.....	2
Brushless Dc motor	4
Servo motor.....	4
Stepper motor	6
Rating & Specification when selecting motor	7
Shooting mechanism & theory.....	9
Catapult.....	9
Linear shooter	9
Flywheel	10
Hardware	11
Bill of Material	12
Hardware setup	13
Wire connection and Pin Mapping	14
Software & system integration.....	15
MD10C Motor driver Control Modes.....	15
Open-Loop vs Closed- Loop control.....	15
Open-Loop Control	15
Closed-loop	15
Encoder integration	16
775-P16 rotary encoder sensor	17
PID control.....	19
Testing Results:.....	22
Problems Encountered & Solutions.....	23
Project Progress Log	25
Conclusion.....	27
Reference	28

Introduction

The ball-launching mechanism is a device designed to propel projectiles with precision and consistency. These systems are widely utilized in competitive robotics such as FIRST Tech Challenge as well as sport automation for basketball, table tennis and football training purpose.

This project serves as technical case study about dc motor and integration of dc motor into ball-launching mechanism. The documentation covers the component selection, electronic control and mechanical design and also record the problems along the way when building the mechanism

Project Objective

To design and build flywheel mechanism capable of launching a rubber ball

Background

Motor types

Motors are foundational actuators in engineering, converting electrical energy and control signals into mechanical rotation through electromagnetism. Many precision robotics and hobbyist projects rely on DC (Direct Current) technology. This section focuses on the types of DC motors commonly encountered in such applications.

Brushed motor

The brushed DC motor represents one of the earliest and most straightforward approaches to generating rotation. Its design consists of a stationary stator (providing a permanent magnetic field) and a rotating rotor (the armature). To maintain continuous spinning, the magnetic field within the rotor must constantly reverse its polarity—a process known as commutation. In these motors, commutation is achieved mechanically using internal brushes that make physical contact with a rotating commutator. As the shaft turns, the brushes switch the current's direction at specific angles, ensuring the magnetic forces always "push" the rotor in the same direction

Brushed motors are favored for their simplicity and low cost. Control is intuitive: reversing the polarity of the power supply reverses the rotation direction, while speed is typically managed via Pulse Width Modulation (PWM) or Lock Anti-Phase techniques. To enhance performance, these motors are frequently paired with encoders to monitor velocity and gearboxes to trade rotational speed for higher output torque

Common examples:



Yellow TT motor

Popular entry-level DC motor encapsulated in a distinct yellow plastic casing. It features an integrated plastic gearbox that provides significant torque at a low speed, making it ideal for beginners.

Operating Voltage: 3V to 6V.

Applications: lightweight 2WD/4WD robotic car, basic line-following robots, and simple DIY mechanisms



SPG30-GR56 (37mm Geared Motor) with encoder

A step up in quality and durability, spur gearbox to increase the torque to 2.1kg.cm with 178 rpm. Use encoder to monitor the speed

Operating voltage: 12v

Application: used in robotics or small-scale conveyor system



The RS-775 is a powerhouse in the brushed motor world. It is often used without a gearbox to take advantage of its high RPM and raw power output. It requires a high-current power source (like a LiPo battery) because its startup current can be very high.

Operating Voltage: 12V to 24V (Capable of 5000+ RPM).

Applications: power tools (like cordless drills), and heavy-duty combat robots

Brushless Dc motor

Brushless DC motors are high-efficiency electrical machines that utilize electronic switching instead of mechanical brushes to create rotational motion. BLDC motor generally have higher efficiency, greater robustness, durability, reduced electrical noise and precise control compared to brushed motor due to its brushless structure

A BLDC motor is fundamentally a three-phase electrical system, consists of 2 main components:

1. Stator: The stationary part which contains copper coils. These coils are energized to become electromagnet.
2. Rotor: The rotating part, which contains permanent magnets.

Unlike a brushed motor use brush as physical contact commutator, a BLDC motor requires an ESC to act as the electronic commutator. ESC provides a precisely timed, three phase electrical signal to magnetize the copper coils sequentially. This alternating activation creates a rotating magnetic field, causing the rotor to spin.

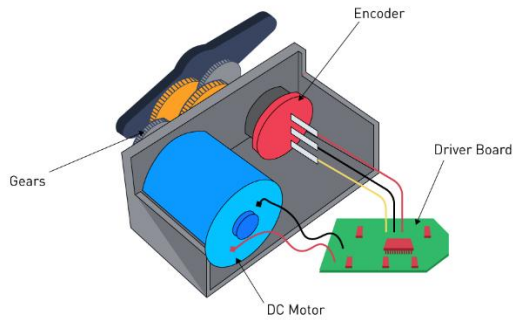
For ESC to know which phase to energize and when to achieve smooth rotation, it must know the exact position of the rotor. There are 2 primary methods to obtain this information. The first method is hall effect sensor which detect the magnetic field from a magnetic ring on rotor shaft. Another method is sensorless control which utilize the back-emf in the operation. The ESC can infer the rotor angle based on the voltage drop and synchronize the control pulses accordingly. Therefore, the speed of rotor shaft can be monitored using back emf without usage of encoder, although it is less precise at very low speeds.

GEPRC GR1404 4500KV



Servo motor

Servomotor is a specialized rotary actuator designed for applications that require precise control of angular position, velocity. Unlike standard dc motors that spin continuously, a servo is engineered to move to a specific angle and maintain that position under load. A servo motor consists of motor, gearbox, output shaft, encoder/potentiometer connected to output shaft and a driver board.



Mechanism of Operation: Closed-Loop Feedback

- i. **Signal Input:** The user provides a control signal, typically via Pulse Width Modulation (PWM), through the signal wire.
- ii. **Comparison:** The internal control board receives this PWM signal and simultaneously reads the current position of the output shaft from the potentiometer.
- iii. **Error Correction:** The board calculates the error—the difference between the desired angle and the actual angle.
- iv. **Actuation:** If an error exists, the board powers the DC motor to rotate the gears in the appropriate direction.
- v. **Steady State:** Once the potentiometer's signal matches the input PWM signal, the control board stops the motor, holding the output shaft firmly in place.



The SG-90 is a widely used, lightweight servo in the robotics community. It features a simple three-wire interface:

Red Wire: Power Supply (+5V)

Brown/Black Wire: Ground (GND)

Orange/Yellow Wire: PWM Signal

In an SG-90, the input PWM signal determines the angle. For example, a 1.5ms pulse typically centers the servo at 90 degrees. If an external force tries to push the output shaft away from this angle, the internal feedback loop will immediately detect the change in the potentiometer's resistance and power the motor to resist the force, maintaining the 90-degree position.

Stepper motor

A stepper motor is a unique type of brushless DC motor that divides a full rotation into a number of equal, discrete steps. A stepper motor moves in precise increments, allowing for accurate positioning without the need for an external feedback sensor (such as an encoder or potentiometer).

Mechanism: Open-Loop control.

The stepper motor functions as an **open-loop** controller. This means that if you command the motor to move 200 steps, it will move exactly 200 steps based on the internal mechanical design, assuming the load is within its torque limits

Stepper motor has fixed resolution. The rotation is divided into a fixed number of steps per revolution. For example, a motor with a 1.8° step angle requires 200 steps to complete a full 360° rotation. Besides, stepper motors excel at staying in one place. When the coils are energized but the motor is not moving, it provides a high "holding torque" that resists external forces.



NEMA 17HS4401

one of the most popular stepper motors in the maker community.

Standard Specs: 1.8° step angle (200 steps/rev) and a rated current of $\sim 1.5A$ to $1.7A$.

Application: Found in CNC machines, desktop scanners, and laser cutters.

Rating & Specification when selecting motor

Speed (RPM)

RPM defines how fast the motor shaft completes a full circle. Generally, higher RPM results in lower torque, while lower RPM offers higher torque.

Application:

High speed: Essential for flywheels to generate enough kinetic energy. These often require speeds of 3000 to 8000+ RPM.

Moderate speed : Ideal for mobile robots (e.g., line followers)

Low Speed: Required for conveyor systems where consistent, slow movement is preferred.

Torque Requirement

Torque is the rotational force that determines how much load a motor can move at a specific distance from the shaft center. Torque is typically measured in kg cm / Nm. A rating of 1 kg·cm means the motor can support or move a 1 kg weight attached to a lever arm exactly 1 cm from the shaft center.

Calculation of mechanism required need to be done based on the condition to determine the torque rating for selecting motor.

Operating voltage

Common DC motor ratings include 3V, 6V, 12V, and 24V. Higher voltage usually translates to higher RPM and torque capabilities. Motor rated voltage must be matched with compatible power supply and a motor driver that can handle the voltage level.

Current consumption

When a motor rotates, it generates Back-EMF, which naturally resists the incoming current. A free spinning motor has very little current. When a load is applied to the motor, the rotor slows down and cause the current draw increase. When load increase the current increase. To the certain point where the load increase and motor stall. The maximum current where the motor stall and couldn't provide torque is called stall current.

Current Type	Description
Free Current	The low current drawn when the motor is spinning freely with no load.
Continuous Current	The average current drawn while the motor is running under its intended load.
Peak (Startup) Current	A brief spike in current when the motor first starts from a stationary position or suddenly changes direction.
Stall Current	The maximum current drawn when the motor is completely stopped (stalled) by a load while still receiving power. This is when the motor provides its maximum torque.



Motor driver must be rated to handle the Stall Current (or at least the Peak Current) to prevent the driver from overheating.

Gearbox

A gearbox is used to increase output torque at the expense of rotational speed.

Plastic gear: Lightweight and cheap; used in Yellow TT motors.

Metal gear: Robust and durable; necessary for high-torque applications but heavier

Spur Gearbox: Simple and cost-effective.

Planetary Gearbox: More durable and compact; provides higher torque density and is less likely to strip under heavy loads

Physical Size & Mounting

Motor size and weight, shaft diameter and length. coupling compatibility

Shooting mechanism & theory

Mechanism exploration

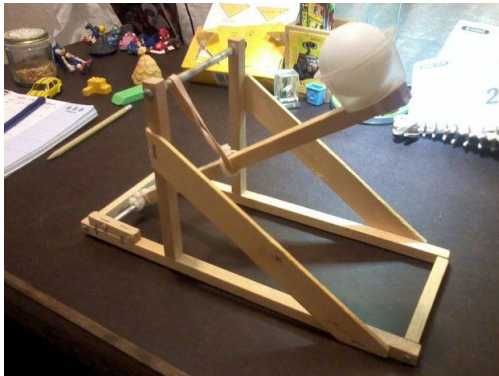
To launch a ball into projectile, we've explored several mechanisms including catapult, linear shooter and flywheel.

Catapult

A catapult utilizes a lever-arm structure to launch the ball. It relies on the sudden release of potential energy. The energy typically stored in stretched springs or rubber bands to propel the ball in an arc.

Pros: High consistency trajectory for fixed-distance shots

Cons: Lack of adjustability and slow cycle time

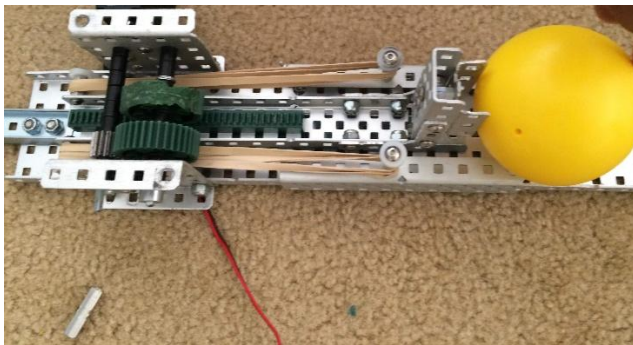


Linear shooter

The linear shooter accelerates the ball along a straight path before releasing it. This motion is often achieved using a high-speed plunger or linear punch actuator.

Pros: rail guide make the prediction and aiming easier, Strength can be adjusted by changing the acceleration

Cons: The mechanism and chassis space required larger compared to other 2 mechanism



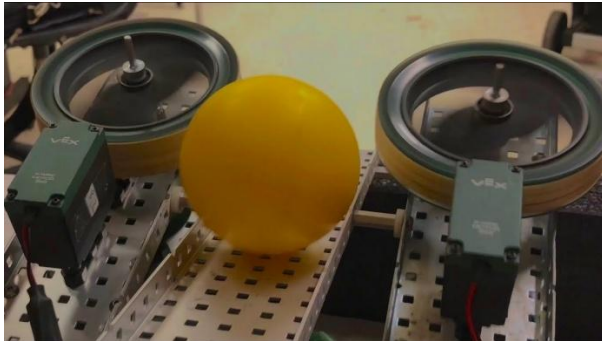
Flywheel

Flywheels operate by transferring kinetic energy from a high-speed rotating mass to the ball.

Effectiveness is determined by the flywheel's RPM (Rotations Per Minute) and its Radius; a larger radius or higher speed allows for more energy transfer.

a. Horizontal double flywheel

This design uses two wheels placed horizontally on either side of ball path, squeezing the ball as it passes through. Kinetic energy is transferred from two sides simultaneously, increasing the potential exit velocity. Because the wheels are horizontal, the ball is less likely to deviate vertically.



b. Hooded flywheel

The hooded flywheel uses a single spinning wheel and a static curved hood or guide rail. The ball is compressed against the hood, allowing it to stay in contact with the spinning wheel for a longer duration compared to double flywheel. This mechanism induces backspin on the ball launched. It is mechanically simpler and lighter.



Hardware

For the initial testing phase of the ball launching mechanism, the hooded flywheel design was selected as the primary propulsion system. This choice was driven by its mechanical simplicity, the relatively low number of required components, and the opportunity it provides to study high-speed DC motor characteristics

The system is designed to test the propulsion of a small rubber ball with the diameter of 5.5cm and weight of approximately 30g.

Core component: RS-775 Motor

RS-775 was chosen for its high kinetic energy potential, which is essential for flywheel application.

Feature	Specification
Voltage Range	12V – 24V
Speed (RPM)	4000 RPM (12V) / 8000 RPM (24V)
Power Rating	80 W
Peak Current	10 A
Stall Torque	2 kg-cm (at 12V)
Shaft Diameter	5 mm

Bill of Material

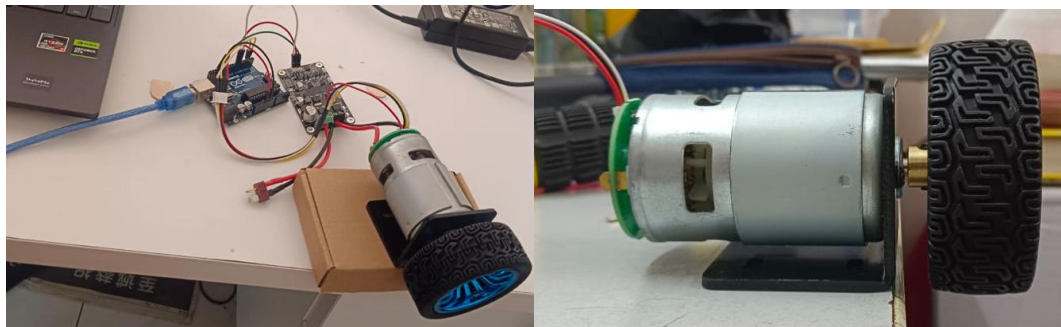
	Items	Spec	Qty	Price
1	RS-775 Motor	12v-24v Peak current: 10A 4000 rpm at 12v, 8000 rpm at 24v 2kg-cm at 12v Weight: 355g Motor shaft: 5mm	1	24.58
2	Thewoodfamer 775-P16 Double Hall Magnet Encoder sensor use to 775 gear motor	Working voltage: 5v Resolution ratio: 16 pulse per turn Diameter of magnetic ring: 5.0mm	1	17.82
3	RS-775 DC Motor Right Angle Mounting Bracket	Material: Mild Steel Material Thickness: 3mm Weight: 105g	1	12.40
4	MD10C Cytron motor driver	Bi-directional control for one brushed dc motor Support motor voltage range: 5v to 30v Continuous current: 13A Peak current: 30A 10 second 3.3v and 5v logic level input No heat sink is required Support locked-antiphase and sign-magnitude PWM operation No reverse Polarity protection	1	55.60
5	Robot rubber wheel	Diameter: 68mm Width: 27mm	1	10.00
6	Hex brass mini wheel Coupling 5mm	Coupling for rubber wheel 5mm for inner shaft diameter Comes with 3 screw to secure the wheel and motor	1	5.50
7	Li-Po Rechargeable RC Tamiya Battery	3s lipo 11.1V 25c discharge rate 1800mAh	1	49.90
8	T-plug Battery connector	16awg	2	3.98

Hardware setup

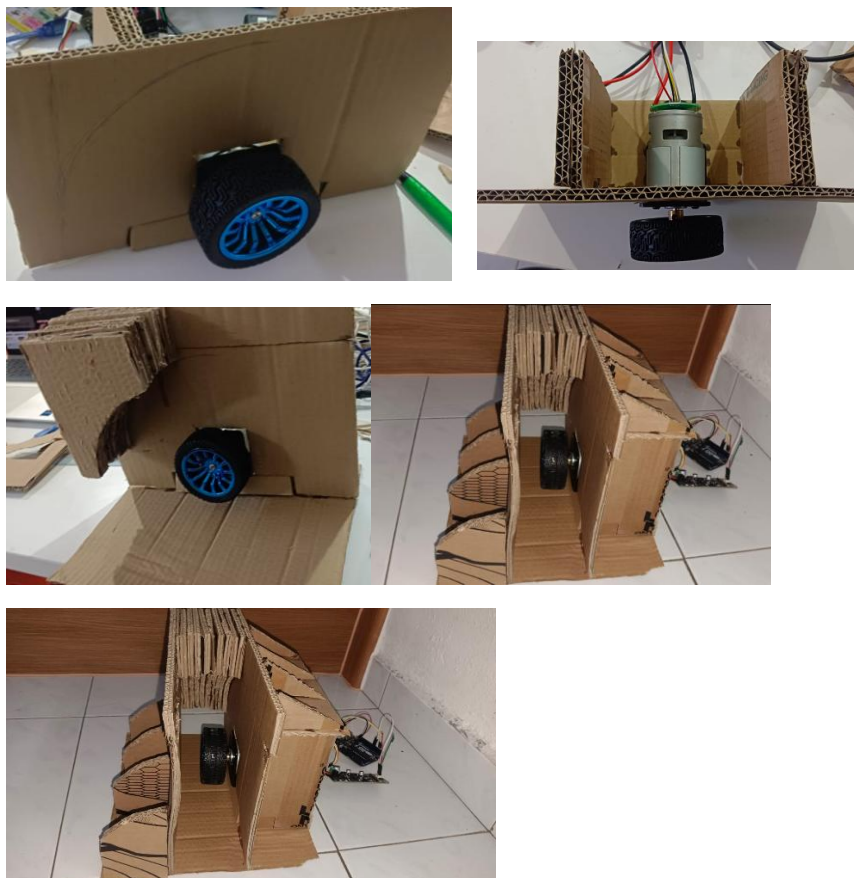
The motor is secured using a mild steel bracket to withstand the vibration of 4000rpm. The 68mm diameter rubber wheel is attached to the 5mm motor shaft via a hex brass coupling, secured with three set screws.

Cytron MD10C motor driver is used between the 3s lipo battery and the motor. This driver is selected for its high peak current (30S), which safely handles the motor's startup and stall conditions.

An Arduino Uno serves as the primary microcontroller. It provides the 5v logic for the encoder and send pwm signals to the motor driver. 775-P16 Hall Effect encoder is mounted at the rear of the motor, providing speed feedback to the Arduino.



Building the motor base and flywheel hood by using cardboard



Wire connection and Pin Mapping

MD10C Motor Driver connections

MD10C Terminal/Pin	Connection Destination	Purpose
DIR Pin	Arduino Pin 6	Direction Control Signal
PWM Pin	Arduino Pin 7	Will be logic HIGH when operating
GND Pin	Arduino GND	Common Ground
Input (V+ / V-)	3S LiPo Battery	Main power supply for the motor
Output (A / B)	RS-775 Terminals	High-power output to the brushed motor

[CAUTION] MD10C does not have reverse polarity protection on the input terminals. Double-check that the positive and Negative leads from the LiPo battery are connected correctly before powering on to prevent hardware damage

775-P16 encoder connection

775-P16 Pin	Connection Destination	Purpose
VCC Pin	Arduino 5V	VCC input
GND Pin	Arduino GND	Common Ground
Phase A	Arduino Pin 2	Hardware interrupt
Phase B	Arduino Pin 3	Hardware interrupt

Pin 2 and 3 on the Arduino uno are specifically chosen because they support external hardware interrupts. Hardware interrupts are utilized to trigger the pulse count because they provide the highest processing priority within the microcontroller. Unlike a standard digitalRead function, which must wait for the main code loop to reach it, an interrupt responds in real-time by instantly pausing the CPU to record the encoder's signal. This ensures a faster, more responsive calculation that is essential for maintaining accuracy at high motor speeds.

Software & system integration

MD10C Motor driver Control Modes

	Pins Required	Logic Description
Sign-Magnitude PWM	2 Pins (PWM & DIR)	The DIR pin controls rotation direction (Logic 0 or 1), while the PWM pin determines the speed (magnitude).
Locked-Antiphase PWM	1 Pin (DIR)	The PWM pin is held at Logic HIGH. The DIR pin receives the PWM signal. A 50% duty cycle represents a stop state, while 0% and 100% represent full speed in opposite directions.

Locked-Antiphase is implemented during the testing because it perform better in :

Smoothness: It provides a smoother transition between speeds.

Low RPM Performance: It is more effective at low speeds because the constant switching in locked-antiphase helps motor overcome static friction more easily than sign-magnitude control

Open-Loop vs Closed- Loop control

Open-Loop Control

In this state, the microcontroller sends a PWM signal (a percentage of total power) without knowing the motor's actual performance.

The Load Problem: When a rubber ball hits the flywheel, the friction causes the RPM to drop. In open-loop, the controller cannot detect this drop or adjust the power to compensate.

The Voltage Problem: DC motor speed is directly proportional to voltage. As the 3S LiPo battery drains (or if a cell is damaged), a PWM signal of 150 might result in 5000 RPM today but only 4000 RPM the next day

Risk: The system cannot detect a stall condition, which could lead to motor or driver failure if the ball gets jammed

Closed-loop

Closed-loop control utilizes the 775-P16 encoder to provide real-time data back to the microcontroller.

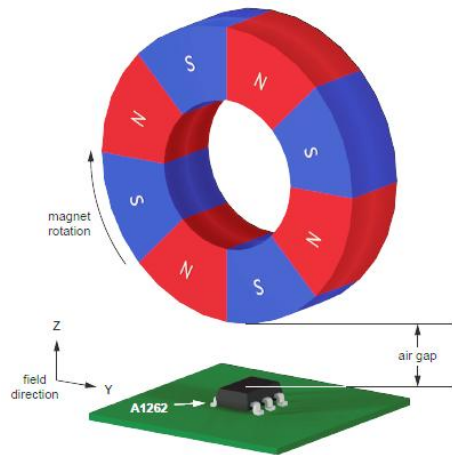
Precision Targeting: Instead of setting a "power percentage," we set a specific Target RPM.

Dynamic Adjustment: When the ball is launched and the flywheel slows down under load, the Arduino detects the RPM decrease via the encoder and instantly increases the PWM signal to return to the target speed.

Reliability: This system ensures consistent launch distances regardless of battery voltage fluctuations or mechanical wear.

Encoder integration

An encoder is a device that converts the angular position or motion of the motor shaft into an electrical signal. It tells the controller where the motor is and how fast it's going. In a closed-loop system, it provide the feedback to microcontroller, providing real time data direction of the motor shaft and how fast it is spinning.



How it work

The Multipole Magnet: Instead of a standard magnet with just one North and one South pole, the encoder uses a specific ring magnet attached to the motor shaft. This ring contains multiple alternating magnetic poles (North, South, North, South...) arranged around its circumference.

The Sensor: One or more Hall effect sensors are mounted on a stationary PCB separated from the magnet by a small air gap. As the motor spins, the alternating magnetic poles pass by the sensor, causing the magnetic field to fluctuate.

The Output: The sensor detects these changes and converts them into a series of digital High/Low voltage pulses. By counting these pulses over a specific timeframe, the microcontroller can calculate the motor's exact RPM.

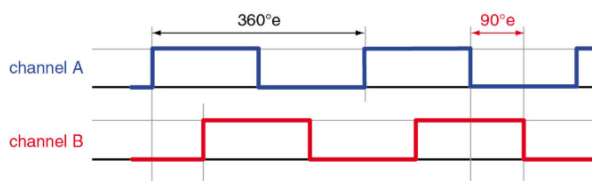
775-P16 rotary encoder sensor

775-P16 is a magnetic incremental encoder that utilizes Hall effect. As the magnetic poles pass the sensor, they trigger high/low voltage pulses.



Quadrature Encoding: Phase A and Phase B

As you can see in graph below, quadrature encoding use two signal wire: phase A and Phase B. These signals are offset by 90 degrees. By checking which phase rise first, the controller knows the direction.



PPR (Pulse Per Revolution)

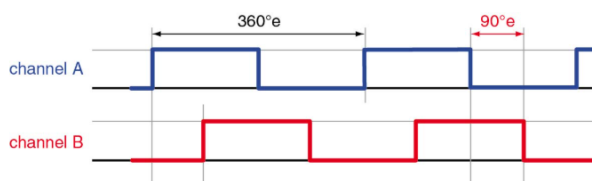
PPR defines the base resolution of the encoder. We can imagine it as the number of magnetic pole pairs on the ring magnet.

It represents the number of full pulse cycles (one HIGH and one LOW) that one channel (e.g., Phase A) will output during one complete 360-degree rotation of the motor shaft.

For the 775-P16 encoder, the base PPR is 16. This means that for every full revolution, Phase A will trigger 16 times.

To calculate speed, we count these pulses over a set sampling time, divide by the PPR, and convert to Revolutions Per Minute (RPM).

Encoder resolution Decoding(1x, 2x, 4x)



While the base PPR is 16, we can increase the precision of our speed calculations purely through software by changing how we "read" or decode the Phase A and Phase B signals.

(Rising: going from LOW to HIGH, Falling: going from HIGH to LOW)

1X Decoding: The microcontroller only counts a pulse when Phase A is Rising (going from LOW to HIGH).

Resolution: 16 counts per revolution.

2X Decoding: The microcontroller counts a pulse every time Phase A is Rising OR Falling.

Resolution: 32 counts per revolution.

4X Decoding: The microcontroller counts a pulse every time Phase A is Rising OR Falling, AND every time Phase B is Rising OR Falling.

Resolution: 64 counts per revolution.

By utilizing 4X decoding in the code via hardware interrupts, a 16 PPR encoder acts like a 64 PPR encoder. This provides the highest resolution, resulting in more accurate RPM readings, faster system updates, and a much more responsive closed-loop PID controller

Example of calculating RPM in code

```
if (currentTime - lastTime >= 50) {  
    noInterrupts();  
    long countCopy = pulseCount;  
    pulseCount = 0;  
    interrupts();  
  
    float instantRpm = (countCopy / (float)ppr) * 1200.0;  
}
```

In sampling time of 50ms, countCopy variable carries the count of pulse in 4x divide by ppr (64) to determine the number of revolutions in this 50ms. The value will then be converted to Revolutions per minute

PID control and Tuning Process

Once the RPM data is obtained from the 775-P16 encoder, it is fed into a Closed-Loop PID (Proportional-Integral-Derivative) system. In this system, we enter a Target RPM, and the microcontroller continuously adjusts the PWM signal to the MD10C driver to minimize the error—the difference between the actual speed and the desired speed.

For this application, a PI (Proportional-Integral) controller was implemented. While a full PID is possible, the Derivative (D) term was omitted as a PI controller is sufficient for motor speed regulation. The Derivative term reacts to sudden, short-term changes; during the motor's rapid acceleration from rest to high RPM, the Derivative term can create large, unstable spikes that complicate the tuning process.

Proportional Gain (K_p)

The K_p term corrects the motor signal based on the current error (Target RPM - Actual RPM).

Function: It provides an immediate push proportional to how far the motor is from its goal.

The Limitation (Steady-State Error): Even with a high K_p value, a gap often remains between the target and real RPM. This is the Steady-State Error. It occurs because as the error becomes very small, the proportional output becomes too weak to overcome internal friction, air resistance, or the load of the flywheel.

Integral Gain (K_i)

To overcome the steady-state error, the Integral (I) term is introduced.

Function: It calculates the accumulation of error over time. If the motor remains below the target for an extended period, the Integral term continuously grows, providing the extra power needed to correct the error and bring the motor to the exact Target RPM.

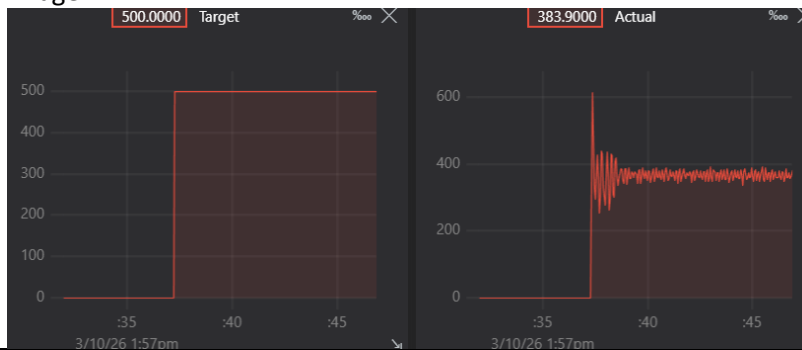
Tuning and Visualization

Arduino Serial Monitor Limitations: Initially, the Arduino Serial Monitor was used to observe the RPM graph. However, the scale was often too magnified, making it difficult to capture enough information to see the big picture of the motor's stability.

The Teleplot Solution: To gain better insights, the project shifted to using VS Code with PlatformIO and the Teleplot extension.

Visualization: Teleplot provides a much larger, clearer picture of the comparison between Target RPM and Real RPM. This allowed for more effective tuning by making it easy to see overshoot and how quickly the K_i term corrected the steady-state error.

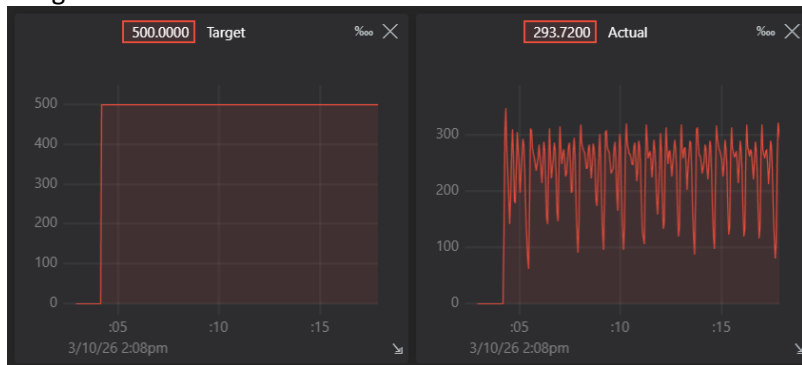
Image1



Kp: 0.2, Ki: 0

Motor reaches ~385RPM; steady state error is observed

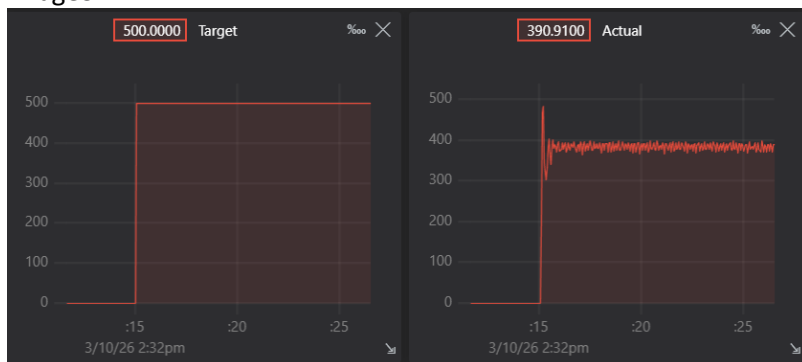
Image2



Kp: 0.1, Ki: 0

High oscillation between 100 to 300 RPM. Less stable than Kp value of 0.2

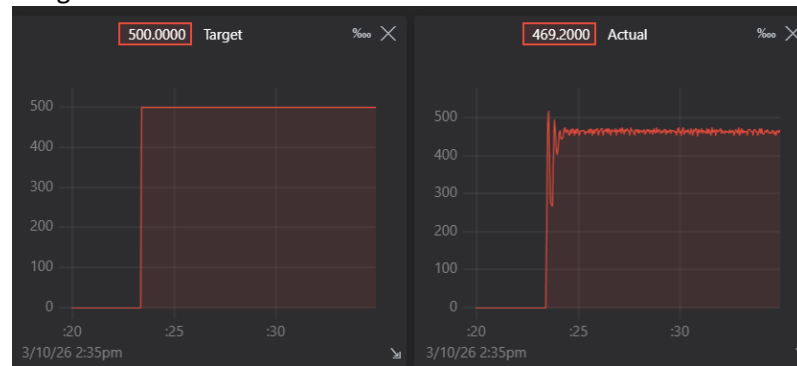
Image3



Kp: 0.18, Ki = 0.15

Added Ki of 0.15. RPM stabilized at 390. Need more Ki value

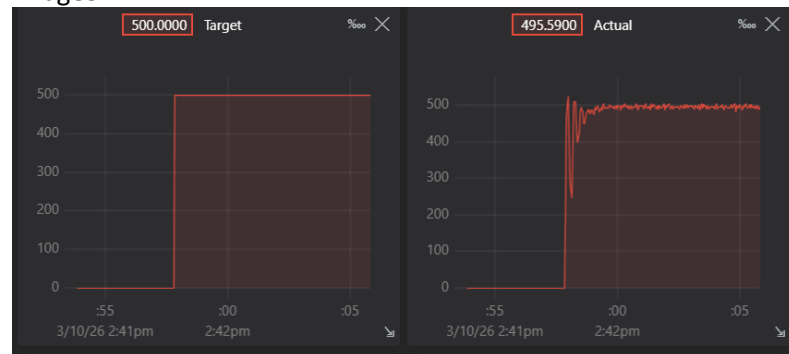
Image4



Kp: 0.18, Ki = 0.35

RPM increased to 470.

Image5



Kp: 0.18, Ki: 0.45

Actual RPM stays at around 495.

Testing Results:

Experimental Setup

Due to the damaged 3rd cell in the original 3S LiPo battery, all final tests were conducted using a 12V 5A AC-DC adapter.

Power Protection: A SkyRC B6 Neo was placed between the adapter and the MD10C motor driver, configured to output a consistent 11.1V at 4.0A.

Control Interface: The Arduino Uno was connected via Teleplot to set target RPMs and monitor real-time velocity feedback.

Procedure: Once the flywheel stabilized at the target RPM, a 30g rubber ball was manually fed into the hooded intake. The horizontal displacement (distance from the launch point to the first point of impact) was measured.

Performance Data: RPM vs Launch distance

The results show a non linear increase in distance as RPM increases

Target RPM (RPM)	Measured Horizontal Distance (cm)
1000	14
2000	20
3000	50

Limitation during testing:

A. Fixed Hood Angle

The current prototype utilizes a fixed-angle hood.

Without the ability to adjust the launch angle (Pitch), the vertical height of the projectile is limited.

B. Electrical Inrush & Protection Trips

The SkyRC B6 Neo frequently identified the motor's startup and sudden braking as a Short Circuit or Outlet Overload. The charger internal safety protection would occasionally cut power to the MD10C to prevent damage.

The RS-775 draws a massive Inrush current spike when transitioning from 0 to high speeds.

Problems Encountered & Solutions

1. Sensing: The 0 RPM Glitter Problem

Problem: While testing the encoder at a 1x resolution (detecting only the rising edge of Phase A), the Serial Monitor frequently showed "0 RPM" even while the motor was clearly spinning.

The Cause: At low resolutions and low baud rates (9600), the sampling window was missing pulses, or electrical noise was interfering with the single-edge detection. This would have caused the PID controller to "panic" and over-correct.

Solution:

Resolution Upgrade: Switched from 1x to 4x resolution, using interrupts on both the rising and falling edges of Phase A and Phase B.

Data Throughput: Increased the baud rate from 9600 to 115200 to ensure the microcontroller could report data as fast as the interrupts were generating it.

2. Latency: Serial Buffer Bottlenecks

Problem: When entering target speeds manually, there was a 1-second delay before the motor responded.

The Cause: The default `Serial.parseInt()` function waits for a long timeout (1 second) to ensure the user has finished typing.

Solution:

Reduced the timeout using `Serial.setTimeout(10)`, limiting the wait to only 10ms. We also decreased the sampling rate from 500ms to 50ms for faster more responsive feedback.

3. PID Tuning

Problem: The standard Arduino Serial Plotter was nearly impossible to tune. It showed a magnified view that looked like sharp zigzag triangles, making it hard to see if the motor was actually stable.

Solution:

Migrated to VS Code + PlatformIO + Teleplot. Teleplot allowed us to see the Big Picture, including the initial startup spike, the oscillation range, and the steady-state error

4. Safety: Uncontrolled Startups

Problem: Upon uploading new code, the motor would sometimes spin back and forth uncontrollably, creating a physical hazard.

Solution:

Remapping the Arduino reset button to act as an emergency stop

6. Power: The Damaged 3S 11.1V LiPo Battery

Problem: The SkyRC B6 Neo charger reported a "Cell Error." Multimeter testing revealed that while the first two cells were at 3.6V each, the total pack was only 7.5V, confirming the 3rd cell was dead/damaged.

Solution:

To avoid the fire risk of a faulty LiPo, we retired the battery for the remainder of the testing phase. We switched to a regulated 12V 5A power supply connected through the B6 Neo to provide a consistent 11.1V to the motor driver for bench testing.

Project Progress Log

Date	Progress
Phase 1: Research & Component Selection	
25/11/2025	Study DC brushed motors, gearbox, and the relationship between torque and RPM.
19/12/2025	Study four flywheel configurations. Identified the Hooded Flywheel as the most efficient setup for transferring energy via increased ball-surface contact area.
25/12/2025	Conducted a comparative study of the JGA and RS-775 series. Pivot: Selected the RS-775 for the launcher due to its high-speed output; JGA series reserved for slower higher torque task, not suitable for flywheel
26/12/2025	Researched 775 motor control applications (e.g., power drills). Finalized the decision to buy the motor and encoder separately
Phase 2: Procurement & Physical Integration	
13/02/2026	Ordered MD10C driver, 2200mAh 3S LiPo, 775 brackets, and mechanical couplings.
23/02/2026	Began assembly. Identified that standard laptop power wires were insufficient for the 10A stall current of the RS-775 motor.
26/02/2026	First Power-On: Integrated 15 AWG high-current wiring and T-plugs. Test bidirectional rotation using manual buttons on the MD10C driver.
Phase 3: hardware testing & signal optimization	
27/02/2026	Implemented Locked-Antiphase PWM via Arduino Uno. Found it smoother than Sign-Magnitude for overcoming initial static motor friction.
03/03/2026	Signal Debugging: Tested 775-P16 encoder at 1x resolution. Encountered Glittering (intermittent 0 RPM readings) caused by electrical noise or low resolution.
04/03/2026	Software Optimization: Upgraded to 4x Resolution (Interrupts on Phase A & B) and 115200 baud rates. Solved the 0 RPM flicker and reduced serial latency to 10ms.
Phase 4: PI tuning & testing	

05/03/2026	Initial PID Test: First closed-loop attempt. Added a low-pass filter to smooth RPM feedback.
10/03/2026	Advanced Tuning: Migrated to VS Code/Teleplot for high-resolution graphing. Finalized PI values: $K_p = 0.18$ and $K_i = 0.45$, successfully eliminating steady-state error.
10/03/2026	Multimeter testing confirmed a damaged 3rd cell in the 3S LiPo battery (7.5V total). Retired the battery to prevent fire risk.
12/03/2026	Configured SkyRC B6 Neo as a power protector (11.1V output) using a 12V 5A adapter to drive the MD10C safely during bench testing.
15/03/2026	Final Validation: Conducted projectile distance tests. Recorded 14 cm at 1000 RPM, 20 cm at 2000 RPM, and 50 cm at 3000 RPM. Identified the fixed hood angle as a limiting factor.

Conclusion

The primary outcome of this project is a functional ball launcher prototype capable of a maximum projectile horizontal distance of 50cm. While this iteration serves as a proof of concept rather than a fully function launcher, it functions as a comprehensive technical case study. The project provided hands-on experience in managing DC brushed motors

In this report, we begin with the foundational study of motor types and selection criteria. We learn mechanical advantage of different launching mechanism and choose hooded flywheel as our mechanism to test and validate. Besides, suitable motor driver and hall magnetic rotary encoder sensor are chosen based on the core component of RS-775 motor.

For the motor control method, we progress from open-loop system, to a closed-loop feedback PI system. We also improve from 1x quadrature to 4x quadrature decoding resolution to achieve a stable velocity profile. The tuning process, visualized through teleplot extension in visual studio code ease us in the process of adjusting Kp and Ki value, ensuring the motor maintain its target RPM and eliminate steady-state error.

From the process, we encountered numerous problems, such as electrical noise in the encoder signals, serial buffer latencies, and hardware failures like a damaged LiPo cell. An initial diagnose will be identified and through online documentation and AI-assisted troubleshooting, we learn to identify the cause for the problems and solve it accordingly.

Certain physical and electrical limitation still remain for future resolution. The fixed hood geometry currently prevents pitch adjustment, limiting the exit angle of the ball trajectory. Furthermore, the high inrush current during startup of motor remains a challenge for skyrc B6 neo trigger protection circuit.

Reference

<https://howtomechatronics.com/how-it-works/how-brushless-motor-and-esc-work/>

https://www.monolithicpower.com/en/learning/mpscholar/electric-motors/servo-motors/basics-of-servo-motor-technology?srsId=AfmBOorcVXKPz35tGuz3HsO-EDPg-bzH0Eao4S1FysxzK4_gr55m-4i

<https://my.cytron.io/tutorial/choose-suitable-motor?srsId=AfmBOoozaZw481shMD2DD-x28yUo2aHSdUrrqbapKikCCt13WBSKPgn-&r=1>

<https://docs.google.com/document/d/1rgQzn-nWn-qcWNnHjDZvIYqUrdCeBQQxXA-TU3BF0AQ/view?tab=t.0>

Appendix

Code

```
#include <arduino.h>

// --- MOTOR PINS ---

const int dirPin = 6;

const int pwmPin = 7;

const int phaseA = 2;

const int phaseB = 3;


// --- ENCODER & RPM ---

volatile long pulseCount = 0;

unsigned long lastTime = 0;

float rpm = 0;

const int ppr = 64;


// --- PID CONSTANTS (Tuning needed!) ---

float Kp = 0.18;

float Ki = 0.55;

float Kd = 0.0;


// --- PID VARIABLES ---

float targetRPM = 0;

float error = 0;

float lastError = 0;

float integral = 0;

float derivative = 0;

float pidOutput = 0;

int finalPWM = 127; // Antiphase stop
```

```

// --- DIRECTIONAL ISRs ---

void encoderA_ISR() {
    if (digitalRead(phaseA) != digitalRead(phaseB)) pulseCount--;
    else pulseCount++;
}

void encoderB_ISR() {
    if (digitalRead(phaseA) == digitalRead(phaseB)) pulseCount--;
    else pulseCount++;
}

void setup(){
    Serial.setTimeout(10);
    pinMode(dirPin, OUTPUT);
    pinMode(pwmPin, OUTPUT);
    pinMode(phaseA, INPUT_PULLUP);
    pinMode(phaseB, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(phaseA), encoderA_ISR, CHANGE);
    attachInterrupt(digitalPinToInterrupt(phaseB), encoderB_ISR, CHANGE);

    digitalWrite(pwmPin, LOW);
    Serial.begin(115200);
    Serial.println("PID Mode: Enter target RPM (e.g., 500 or -500)");
}

void loop() {
    unsigned long currentTime = millis();

    // --- PID CALCULATION (Every 20ms) ---

```

```

if (currentTime - lastTime >= 50) {

    noInterrupts();

    long countCopy = pulseCount;
    pulseCount = 0;
    interrupts();

    float instantRpm = (countCopy / (float)ppr) * 1200.0;
    // Low-pass filter (Smooths out the 47 RPM jumps)
    // 0.7 = how much to trust the NEW reading. Adjust 0.1 to 0.9.
    rpm = (rpm * 0.7) + (instantRpm * 0.3);

    // 2. PID Logic
    error = targetRPM - rpm;
    if (abs(pidOutput) < 120){
        integral += error*0.05;
    }

    integral = constrain(integral, -100, 100);
    derivative = (error - lastError) / 0.05;

    pidOutput = (Kp * error) + (Ki * integral) + (Kd * derivative);

    finalPWM = 127 + (int)pidOutput;
    finalPWM = constrain(finalPWM, 0, 255);

    analogWrite(dirPin, finalPWM);

    // 4. Debugging

```

```
// --- CORRECT TELEPLOT FORMAT ---  
Serial.print(">Target:");  
Serial.print(targetRPM);  
Serial.println(" g"); // Must be println to end the command
```

```
Serial.print(">Actual:");  
Serial.print(rpm);  
Serial.println(" g"); // Must be println to end the command
```

```
lastError = error;  
lastTime = currentTime;  
}
```

```
// --- SERIAL INPUT ---  
if (Serial.available() > 0) {  
  char c = Serial.peek();  
  if (c == 's' || c == 'S') {  
    Serial.read();  
    targetRPM = 0;  
    integral = 0;  
    digitalWrite(pwmPin, LOW); // Kill power  
    Serial.println("STOP");  
  }  
  else if (isDigit(c) || c == '-') {  
    digitalWrite(pwmPin, HIGH); // Wake up driver  
    targetRPM = Serial.parseFloat();  
  }  
  else {  
    Serial.read();  
  }  
}
```


}

}

}